LSTNet: long- and short-term patterns in multivariate time series

Electricity use repeats every day and every week, and a forecast should know it. Lai et al. (2018), translated from their code and tested against it.

Tested against	the authors' PyTorch code (`laiguokun/LSTNet`, commit `093c4dc`: `models/LSTNet.py`, `main.py`, `utils.py`, `Optim.py`), on three artificial datasets, with their weights, batch order and settings; and one cell of Lai et al. (2018), Table 2 (p. 101): Exchange-Rate, horizon 3.
Missing in Julia	LSTNet-Skip as the authors' code has it. FluxArchitectures.jl has a model called `LSTnet`, but a different one (box below).
Notebook	**Compose** › **Workflows** › **Browse Analytical Workflows**: *LSTNet: long- and short-term patterns in multivariate time series* (`LSTNet.ipynb`)
Result	with the same weights, the forecasts to 1.8×10^{-7}; with the same batches, 3 epochs of training steps to 1.1×10^{-6}. Trained from scratch, Exchange-Rate at horizon 3: RSE 0.0256 and CORR 0.9706 against the article's 0.0226 and 0.9735.

The problem

Many series are measured side by side: the consumption of hundreds of electricity clients, the occupancy of hundreds of road sensors, the output of solar plants. They share patterns on two scales, the hours of a day and the days of a week, and they move together. A forecast should use both: the short-term dynamics of the last few hours and the long-term pattern of the same hour on earlier days. Lai et al. (2018) built a network for it, LSTNet, and compared it with eight methods on four datasets (Table 2). Their code, public, is the reference here.

The method, as the code has it

For *m* series and a window of the last 168 observations of each, the network (`network`) adds two forecasts of all *m* series at once:

- **The neural part.** A convolution of 6 steps across all the series, 100 filters (50 for Exchange-Rate), ReLU; a GRU over its 163 outputs; a second GRU, the *recurrent skip*, over the steps 24 apart, one run for each hour of the day, over the last 6 days; the last hidden states of both, joined, mapped to the *m* forecasts by a dense layer. Dropout 0.2 after the convolution and after each GRU.
- **The highway.** An autoregression on the last 24 values of each series, with the same weights for every series, for the scale, which the neural part is insensitive to.

Each series is divided by its largest absolute value; the first 60 per cent of the times train, the next 20 validate and the last 20 test (`Split`). Training (`forecast`) minimises the absolute error in the units of the data, with Adam (learning rate 0.001), batches of 128 and the gradient's norm clipped at 10, for 100 epochs, and keeps the epoch with the smallest validation RSE. `evaluate` gives the article's RSE (root mean squared error over the standard deviation of the data) and CORR (the mean correlation of each series' forecasts and values). `SETTINGS` holds each dataset's hidden sizes and output function, from the authors' scripts, and `table2` computes a dataset's columns of Table 2, saving each finished horizon and, after every epoch, the training, so a stopped run continues.

The network runs on any GPU that Julia's `gpu_device()` finds (CUDA, AMDGPU, Metal, oneAPI) once its package is loaded, with Zygote for the gradients, and otherwise on the CPU, with Enzyme.

The code, not the paper

Where the article and the code differ, the translation follows the code:

Point	The code (and this package)	The article
GRU activation	tanh (PyTorch's `nn.GRU`)	ReLU (Section 3.3)
Skip periods	$\lfloor (168-6)/24 \rfloor = 6$: the code divides without rounding, 6.75, and fails with its own defaults; the first author's fix (issue #2 of the repository) takes the whole part	not stated
Loss	absolute error on the rescaled data, always: `--L1Loss False` in `stock.sh` is read as true, since Python's `bool("False")` is true	chosen on the validation set (Section 3.7)
Settings	one script per dataset: hidden sizes, and the output function (identity, or sigmoid for Traffic)	tuned by grid search (Section 4.4)
Horizons	the scripts run one horizon each (24 for Electricity, the default 12 for the others); we vary only `--horizon`	3, 6, 12, 24
RSE	divided by `torch.std` of all test values times $\sqrt{(r-1)/r}$, r the number of rows	the standard deviation of the test values
CORR	averaged over the series not constant in the test period; the code's float32 check can keep a constant series with correlation 0, which we leave out	the same, without the check

What the tests show

The cross-tests run the authors' code (with one-line patches for a current PyTorch, none changing the model) and the package on three artificial datasets of 1200 times: 6 seasonal series, 6 series with a trend and scales from 1 to 1000, and 5 series with a constant stretch; horizons 3 and 12, so six cases; window 48, kernel 6, skip 12, hidden sizes 16, 16 and 4. Every case, the largest difference:

Test	What is compared	Largest difference
A	the authors' initial weights: forecasts of the test times; their RSE; their CORR	1.8×10^{-7}; 1.4×10^{-7}; 6.6×10^{-8}
B	the same weights, their batch order, dropout 0, 3 epochs (18 steps): every gradient norm; every validation RSE; the final weights (relative)	1.1×10^{-6}; 2.5×10^{-6}; 8.9×10^{-5}

A and B run in `Pkg.test`, on a GPU if one is loaded, and need no Python. C trains both codes from scratch, 5 seeds each, with their own random numbers, so only the spread can be compared. Test RSE, lowest to highest:

Case	Authors' code	This package
seasonal, $h = 3$	0.1761 to 0.1860	0.1745 to 0.1838
seasonal, $h = 12$	0.1906 to 0.2027	0.1896 to 0.2015
trend, $h = 3$	0.1015 to 0.1092	0.1044 to 0.1162
trend, $h = 12$	0.1153 to 0.1200	0.1166 to 0.1180
constant, $h = 3$	0.2911 to 0.3688	0.2865 to 0.4606
constant, $h = 12$	0.2743 to 0.3034	0.2564 to 0.3634

Every pair of ranges overlaps. The CORRs agree as well, except for the constant series, where the authors' code reports 0.77 and the package 0.96 to 0.97: of its 5 series, the constant one is counted with correlation 0, and $0.96 \times 4/5 = 0.77$.

D gives both codes everything but the code itself, for all 100 epochs, and still their results part: the box below says why, and what it means for any replication of a trained network.

Same code, same data, same random numbers: why the numbers still part

Cross-test D gives both codes everything that could make them differ, except the code itself: the artificial data, the authors' initial weights, their batch order for all 100 epochs (600 steps), and dropout 0, so no random numbers are drawn at all. Both train in float32: the authors' data, scale and weights are float32 (`torch.zeros`, `.float()`, PyTorch's default), and ours are `Float32` throughout. Still, their numbers part after 10 to 51 epochs:

Case	Epoch kept	Test RSE	Test CORR	Epoch valid RSE
	Julia / authors	Julia / authors	Julia / authors	**0.1 % apart**
seasonal, $h = 3$	100 / 100	0.1783 / 0.1800	0.9788 / 0.9787	14
seasonal, $h = 12$	98 / 92	0.1909 / 0.1922	0.9755 / 0.9750	10
trend, $h = 3$	99 / 99	0.1126 / 0.1094	0.8123 / 0.8129	51
trend, $h = 12$	93 / 79	0.1182 / 0.1165	0.7517 / 0.7498	19
constant, $h = 3$	32 / 38	0.4426 / 0.4421	0.9616 / 0.9618	15
constant, $h = 12$	98 / 67	0.4217 / 0.4174	0.9696 / 0.9689	21

Not the code: rounding

Every arithmetic operation in float32 rounds its result to about 7 significant digits. PyTorch and Julia do the same operations in

a different order (sums of many terms, the matrix products of BLAS), so each rounds a little differently, at every step. After one step the two codes' weights are 2.9×10^{-8} apart (relative distance), after 100 steps 5.9×10^{-7}, after 230 steps 1.8×10^{-6}: a drift, still far below any digit printed.

The switch that amplifies it: ReLU

The convolution's outputs pass through a ReLU, which is a switch: an input just above zero passes, one just below gives zero, and so does its gradient. When an input lies closer to zero than the gap between the two codes, the codes put it on opposite sides, and from then on they compute different gradients. We traced the trend case, $h = 3$, step by step: at every step the 3022 pre-activations that PyTorch had within 10^{-4} of zero, compared with Julia's.

Step	Event	Weight distance
1–230	rounding only	2.9×10^{-8} to 1.8×10^{-6}
232	first flip, filter 5: Julia $+2.6 \times 10^{-6}$, PyTorch -1.2×10^{-7} (8 positions)	1.8×10^{-6}
249	flip, filter 5 (5 positions)	1.8×10^{-6}
275	flip, filter 5: Julia $+1.1 \times 10^{-5}$, PyTorch -5.2×10^{-5} (10 positions)	3.7×10^{-6}
280–300	more flips, at steps 284, 285, 286, 287, 291, …	8.8×10^{-6} to 1.3×10^{-4}
400, 600		9.2×10^{-3}, 1.6×10^{-2}

The positions that flip at one step are one stretch of the data seen by several overlapping windows of the batch (data rows 390, 82 and 517 at steps 232, 249 and 275), not a dead filter: filter 5's weights are as large as the others'. Why the flip at step 275 starts the growth, and those at 232 and 249 leave the distance where it was, we have not established.

The authors' code against itself

How much of this is the translation? We ran the authors' code twice with everything the same but every initial weight moved by one float32 rounding unit (about 10^{-7} relative): up, down, and three times up or down at random. In lockstep, on the seasonal case, $h = 3$, the two runs are 8×10^{-8} apart at step 1 and 2.6×10^{-7} at step 115; at step 116 an input 5×10^{-7} from zero flips, and the distance is 1.3×10^{-6}, then 2.1×10^{-4} at step 130, 1.8×10^{-3} at 150 and about 8×10^{-3} at 600. For the trend case the first flip comes at step 452; for seasonal, $h = 12$, none comes in 600 steps (6.1×10^{-7} at the end). Over the five nudged runs, the test RSE of the epoch kept is

Case	Test RSE of the authors' code, 5 nudged runs	Epochs kept
seasonal, $h = 3$	0.1800 to 0.1802	84 to 100
seasonal, $h = 12$	0.1922 in all five	92
trend, $h = 3$	0.1090 to 0.1108	95 to 99
trend, $h = 12$	0.1165 in all five	79
constant, $h = 3$	0.4421 to 0.4422	32 or 38
constant, $h = 12$	0.4172 to 0.4197	67 to 85

So one rounding unit, once, moves the authors' own results by up to 1.6 % (trend, $h = 3$) and the epoch kept by up to 18. Julia against PyTorch differ by rounding at every operation, so they part sooner and in every case, by up to 2.9 % (trend, $h = 3$). After parting, neither run gets worse: they settle at different solutions, equally good.

What this means for a replication

- **Test a translation with the same random numbers, step by step.** Only there can it agree to the last digits: here the forecasts to 1.8×10^{-7} (A) and every gradient norm of 3 epochs to 1.1×10^{-6} (B).
- **Measure the original against itself.** A one-unit nudge of its starting point shows how far rounding alone moves its results; a translation that differs by no more has nothing left to explain.
- **A single published run carries this spread.** Table 2 reports one run per cell, so its values are uncertain by at least as much, before any change of seed, hardware or library.
- **The epoch kept is fragile.** Near the end the validation RSE is nearly flat, so the smallest moves from one epoch to another with the least change.

The traces are reproducible: `crosstest/divergence/lockstep.py` (the authors' code against itself), `trace.py` and `trace.jl` (Julia against PyTorch, step by step).

Table 2

Exchange-Rate, $h = 3$	RSE	CORR	Epoch kept	Training	
This package	0.0256	0.9706	97 of 100	1729 s	this Mac's GPU (Metal), seed 54321
Lai et al. (2018), p. 101	0.0226	0.9735			

One training from scratch, with the authors' settings for this dataset (`stock.sh`: hidden sizes 50, no output function). The RSE is 13 per cent higher, the CORR 0.003 lower. The cross-tests settle that the package computes what the authors' code computes; a single run from scratch cannot settle more. The article reports one run per cell, its seed is not known (54321 is the code's default; the scripts set none), its hidden sizes were tuned by a grid search the scripts do not record, and the box above shows that one rounding unit moves the authors' own result by up to 1.6 per cent and the epoch kept by up to 18. PyTorch and Julia draw different numbers from the same seed, so the article's run cannot be repeated exactly, in either language.

The other 15 cells were not run: Solar-Energy, Electricity and Traffic have 137, 321 and 862 series, and each cell is 100 epochs. They are tests of the package (`Pkg.test("LSTNet"; test_args = ["solar"])`), not run by default, for a machine with an NVIDIA GPU; `table2` with a `folder` can be stopped and continued.

Why not FluxArchitectures.jl?

A registered Julia package already has a model called LSTNet: `LSTnet` in FluxArchitectures.jl (S. Dobberschütz), which Pollis uses in its neural time-series panel. It follows the idea of the paper, not the authors' code, and it is a different model:

	Authors' code (and this package)	**FluxArchitectures `LSTnet`**
Forecasts	all *m* series at once (`Linear` to *m* outputs)	one series, the first column (`Dense` to 1)
Convolution	kernel of 6 time steps across all series, sliding along the 168-step window	kernel as long as the whole window: one vector per sample
The GRU runs over	the 163 convolution outputs inside each window	consecutive samples, carrying its state from one sample to the next
GRU activation	tanh (PyTorch's `nn.GRU`)	ReLU, as in Section 3.3 of the paper
Skip component	a second GRU over the skip periods, one run per phase; final states concatenated	one GRU fed the hidden state from *p* steps back
Autoregressive part	the last 24 values of each series, the same weights for every series	the last time step only, one weight per series, summed into one output
Dropout	0.2	none
Scaling	each series divided by its largest absolute value	*z*-scores over the whole series, with no training/validation/test split
Training	L1 loss on the rescaled data, batches of 128, gradient norm clipped at 10	mean squared error, the whole sample in one batch (the package's example)

Since it forecasts a single series, it cannot give Table 2, whose RSE and CORR are taken over all series, and the cross-tests of this package cannot be run on it: its weights have other shapes, so there is nothing to load the authors' weights into. It requires Flux 0.13 and Zygote older than 0.6.44.

Further considerations

- **LSTNet-Attn and the baselines.** The authors' code has LSTNet-Skip only; the attention variant and the eight methods of Table 2 would be implementations of our own, not translations, and are not included.
- **The rest of Table 2.** Fifteen cells, on a CUDA machine, and several seeds per cell, would give the spread a single published run hides.
- **Upstream.** The cross-tests, with the authors' weights in `reference.json`, are a ready-made test for any Julia implementation of LSTNet.

References

Lai, G., Chang, W.-C., Yang, Y. and Liu, H. (2018). Modeling long- and short-term temporal patterns with deep neural networks. In *SIGIR '18: The 41st International ACM SIGIR Conference on Research and Development in Information Retrieval*, 95–104. doi:10.1145/3209978.3210006

Code: github.com/laiguokun/LSTNet, commit 093c4dc, and issue #2 on the skip periods (31 July 2018).

Data: github.com/laiguokun/multivariate-time-series-data, read into memory and not saved, since the repository has no licence.

Methods written and tested with Claude Code (Anthropic) in Pollis, from the published papers and their data.

© 2026 Antonio Saragga Seabra · Licensed under CC BY-SA 4.0 · Draft, 15 October 2026

contact@pollis.org